

OptArrow Design Document

Introduction

The OptArrow optimization integration engine is designed to facilitate connections between optimization clients and solver backends using a high-performance, stable transport layer. The current runtime is centered around Python and Julia backends, with additional interaction capabilities available via a lightweight MATLAB client interface. The primary objective is to tackle optimization problems, particularly Linear Programming (LP) and Quadratic Programming (QP). The following documentation provides an in-depth view of how architectural components are implemented, focusing on mechanisms, interfaces, and the transformation of data representations within the system based on validated claims.

Stakeholder Engagement Mechanisms

Feedback Collection

OptArrow currently manages stakeholder input through GitHub Issues, serving as the central mechanism for contributions and problem reporting. This approach ensures streamlined communication with stakeholders, whose feedback can directly influence future iterations of the product. However, the absence of a formal requirements traceability system marks an area for future development. This advancement would enable connections from stakeholder feedback through to decisions and implementations, promoting comprehensive requirement management.

Architectural Decision Making and Evolution

Use of Apache Arrow for In-Memory Data Transfer

OptArrow has strategically employed Apache Arrow for in-memory data transfer due to its capability to support high performance and cross-language compatibility. Specifically, data transitions within OptArrow involve transforming Python dictionaries into Arrow IPC (Inter-Process Communication) format. This intermediate format allows seamless handoff to the Julia environment, where the computations necessary for optimization tasks can be executed efficiently.

HiGHS Solver Integration

The HiGHS solver was selected as the backend due to its free and open-source nature, and its popularity among the scientific community for solving LP and QP problems. OptArrow integrates HiGHS directly, allowing for scalable computational capabilities. The solver receives input typically formatted through an Arrow IPC mechanism, ensuring that data is promptly and accurately processed for optimization calculations.

Iterative Architecture Evolution

The architectural design of OptArrow evolved iteratively rather than through a pre-defined roadmap. This iterative process enables flexibility, allowing the incorporation of insights from stakeholder feedback and technological assessments. OptArrow's architecture is

continuously refined, accommodating necessary modifications across development cycles.

Historical Context Involving COBRA Toolbox

Historically, the COBRA Toolbox provided a foundational architectural approach that influenced OptArrow's evolution. This reference highlights the adaptability and lessons learned from previous developments, serving as critical considerations in determining the current architectural components and configurations.

Design Alternatives and Technology Evaluation

Absence of Alternative Protocol Trials

In selecting Apache Arrow for data transfer, no comprehensive trials involving alternative data-transfer protocols were conducted initially. The absence of this comparative analysis represents a potential future opportunity to validate Apache Arrow's selection against other protocols, ensuring it meets optimal performance criteria under various operational scenarios.

Exploration of Multiple Implementation Routes

Throughout OptArrow's development, multiple implementation approaches were explored and are documented in the GitHub history. This iterative experimentation is a testament to the robustness and adaptability of the system, as developers evaluated different routes to address challenges and optimize functionalities.

Process Governance and Maturity

Lack of Formal Architecture Governance

OptArrow currently operates without a formalized architecture governance framework. Establishing such a process would enhance the transparency and scalability of architectural changes. The implementation of documentation for architectural modifications and formal governance processes would represent significant strides toward maturity.

Need for Maturing Requirements Documentation

There is a recognized need for maturing the requirements documentation process within OptArrow. Developing formal documentation and traceability procedures will provide a framework for consistent decision-making and reliable development activities, ultimately supporting robust project outcomes.

Recommendations for Future Development

Enhanced Traceability Processes

To improve requirements management, the implementation of an enhanced traceability process is recommended, aligning stakeholder needs with decision-making and implementation records. The use of industry-standard practices like Architecture Decision Records (ADRs) would significantly contribute to achieving this goal.

Formal Review Process for Stakeholder Feedback

A formal process for reviewing stakeholder feedback is essential for aligning the project with user expectations and ensuring long-term success. Developing such a process would facilitate actionable insights, allowing OptArrow to adapt effectively to the dynamic needs of its user base.

Summary

This design document captures the current state of OptArrow's architecture and identifies areas for future improvement. By grounding development in validated claims and insights from historical evidence, OptArrow aims to continue its evolution as a robust optimization engine featuring enhanced scalability, process clarity, and stakeholder integration. Future efforts will prioritize formalizing governance structures, improving traceability processes, and exploring alternative technologies to align with industry standards in optimization software development.